CSE 554 Machine Learning Systems

Lecture 9 - Memory Management

Baris Kasikci

BE BOUNDLESS

Class Overview

1. GPU Architecture and Programming
2. Transformers (Architecture and Implementation)
3. Performance Modeling
4. Quantization
5. **Memory Management**
6. NVIDIA Blackwell Architecture
7. Sparsity
8. Mixture of Experts (MoE)
9. Batching
10. Speculative decoding
11. Parallelism
12. End-to-end serving
13. Agentic systems

Calculating the KV cache size

- Num KV heads
- Head Dim
- 2 (stores both K and V)
- Stype (2 Bytes)
- Seqlen
- Layer

Batch Size is limited by the KV Cache

- Serving Llama3-8B model using H100 -> 80GB in total
- Model weights 2*8=16GB
- Activations are negligible
- 1024 input, 1024 output
- Each request's KV cache
  - (1024 + 1024 / 2) * 2 * 2 * 8 * 128 * 32 / 1024 / 1024 = 192 MB
- Max possible batch size
  - (80 – 16) / (192 / 1024) = 341
    - Recall: batch size of 333 was needed to drive H100 to the compute-bound regime

We should fully utilize the GPU memory capacity to enlarge the batch size

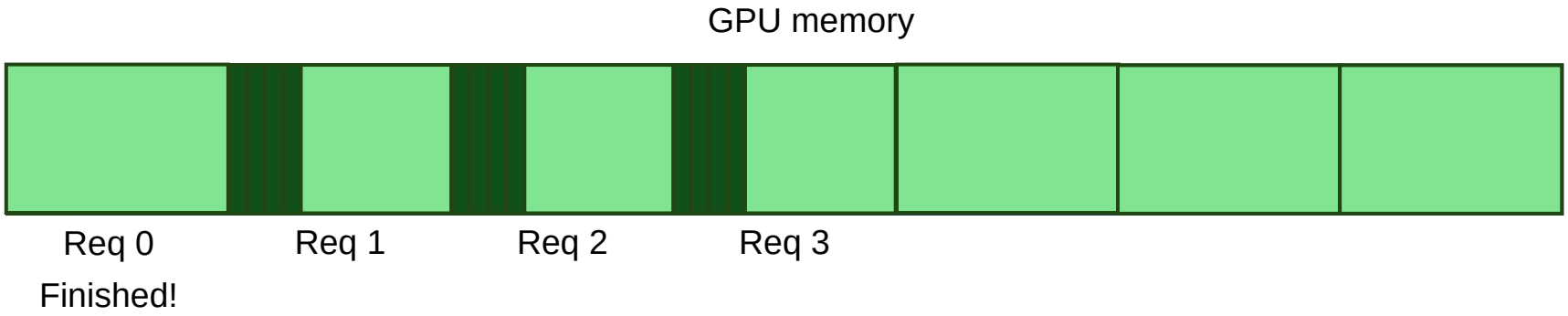
Requests with different lengths

- Serving Llama3-8B model using H100 -> 80GB in total
- Model weights 2*8=16GB
- 1024 input, 50% 1024 output, 50% 4096 output
- Each request's KV cache
  - Min = (1024 + 1024 / 2) * 2 * 2 * 8 * 128 * 32 / 1024 / 1024 = 192 MB
  - Max = (1024 + 4096 / 2) * 2 * 2 * 8 * 128 * 32 / 1024 / 1024 = 384 MB

How to allocate the KV cache for requests with different lengths? (Note that before you are done decoding, you don't know the decoding length)

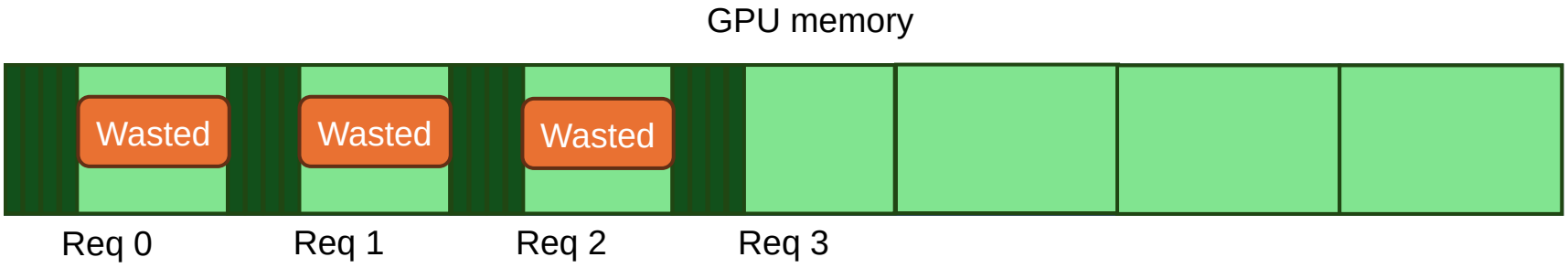
Allocation Method 1 – Fixed Max seq_len

- Allocate the KV cache for each request using the max sequence length the model supports



Allocation Method 1 – Fixed Max Seq len

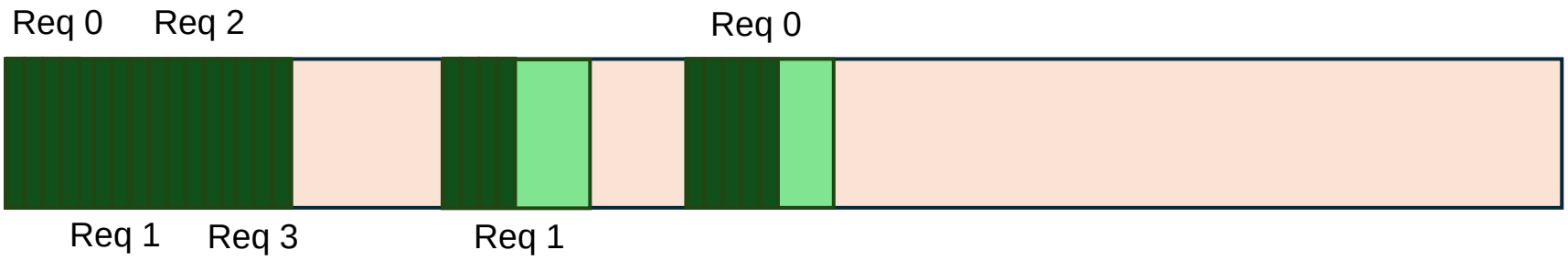
- Allocate Max seqlen -> 4096
- Max batch size = (80 – 16) / (384 / 1024) = 170



Low utilization due to internal fragmentation

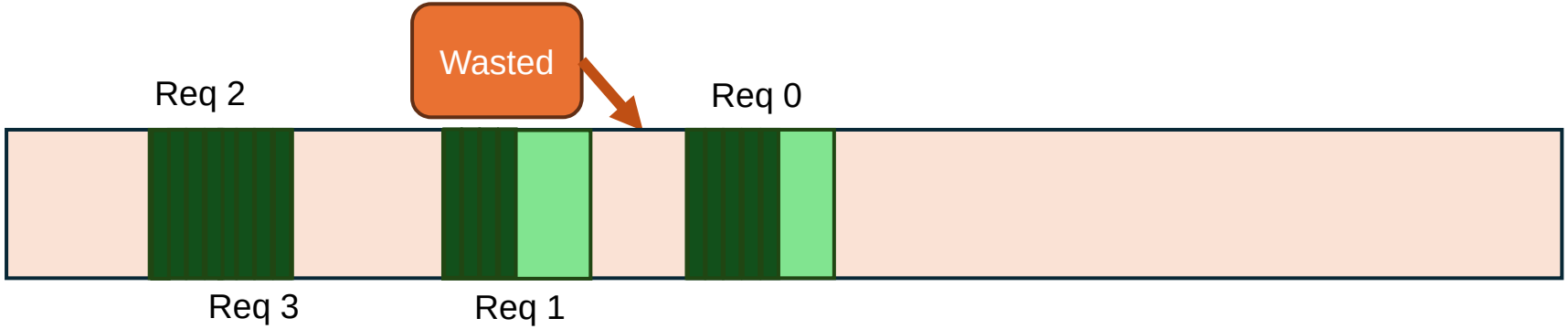
Allocation Method 2 – std::vector-style Allocation

- Start with a small size, when fully occupied, double the size



Allocation Method 2 – std::vector-style Allocation

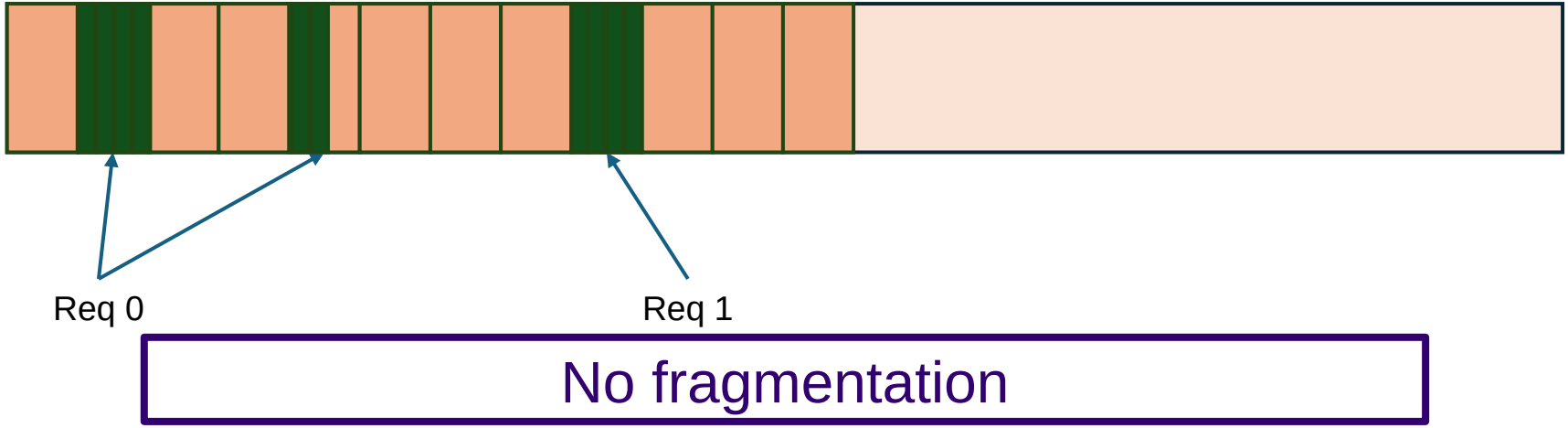
- Now the waste inside a request is “well” controlled (avg 75% utilization)
- Additional copy is acceptable



Low utilization due to external fragmentation

Allocation Method 3 – PagedAttention

- Chunk the global space into small pages
- Each request picks some page as the KV cache
- Each page is large enough for good bandwidth utilization
- Page = 16 tokens' KV = 16*2*2*8*128 = 64 KB+



Kwon et al., Efficient Memory Management for Large Language Model Serving with PagedAttention
https://arxiv.org/abs/2309.06180
+8: number of KV heads

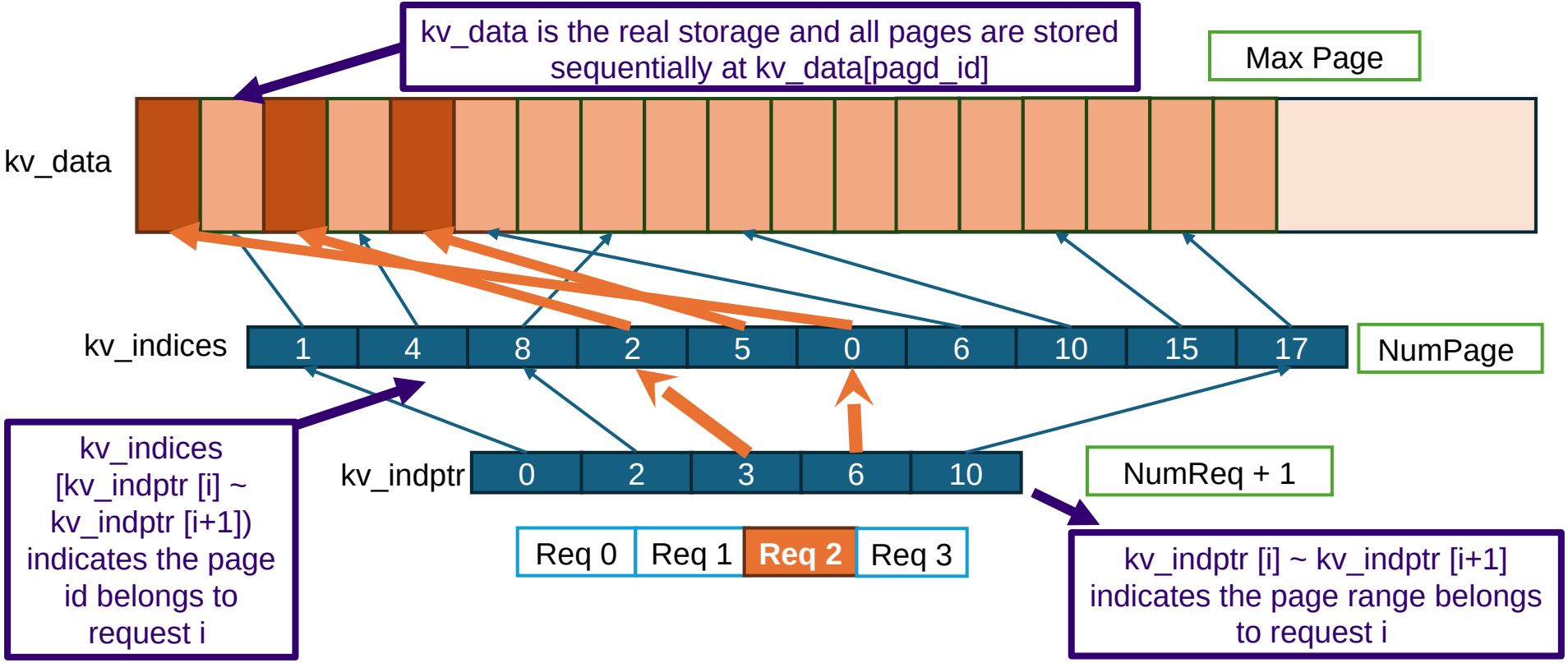
Allocation Method 3 – PagedAttention

- Due to the different number of pages per request, a basic page table is unbalanced
  - The page table also needs to be moved back and forth between the CPU and the GPU t every iteration.

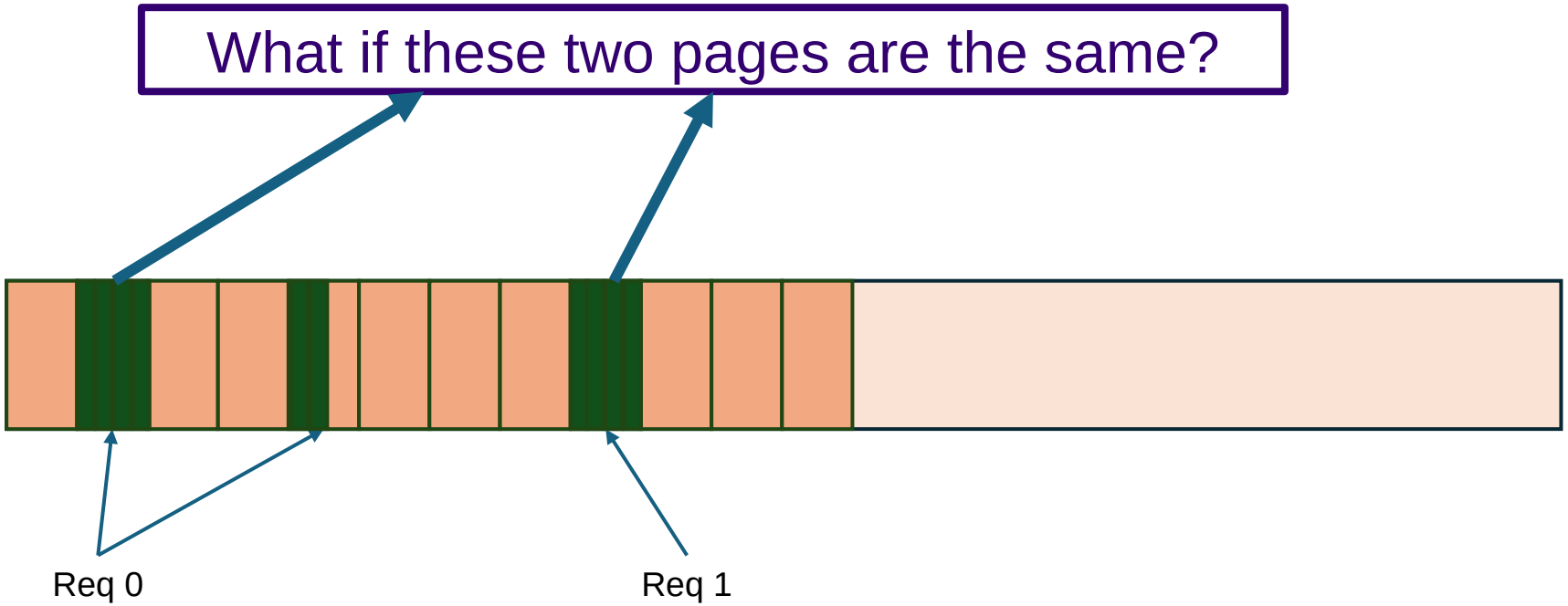
Request ID	Page ID			
0	1	4		
1	8			
2	2	5	0	
3	6	10	15	17

How to handle page tables with variable-length entries without padding?

Multi-Level Page Table

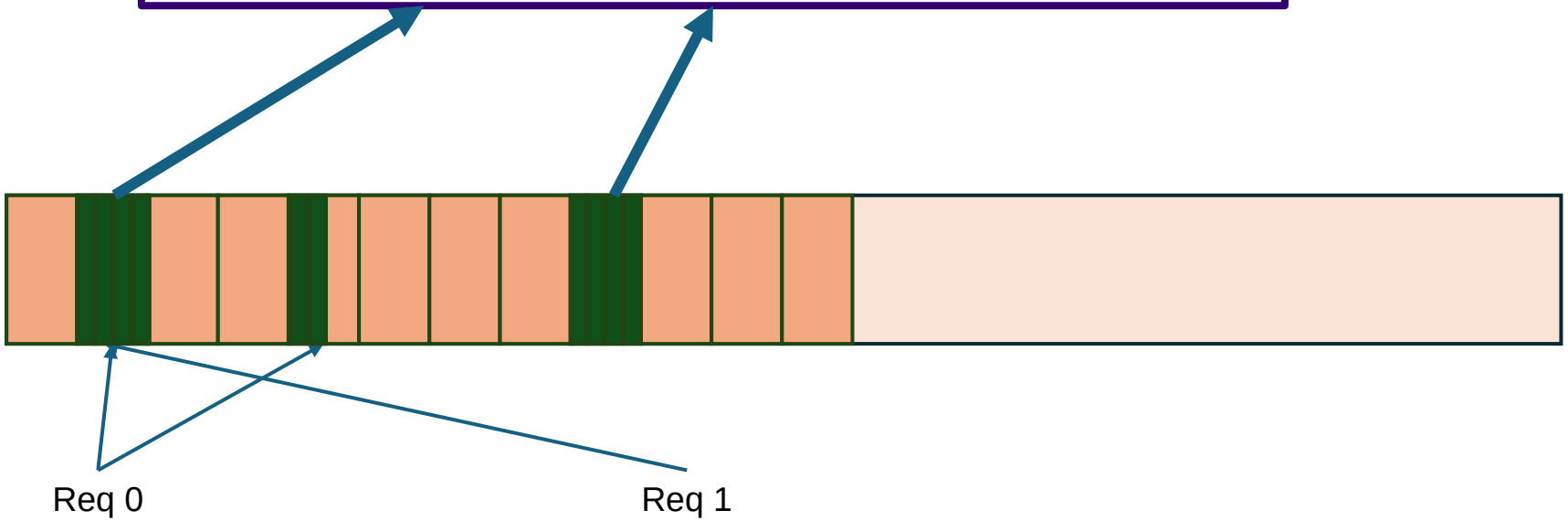


Prefix Sharing



Prefix Sharing

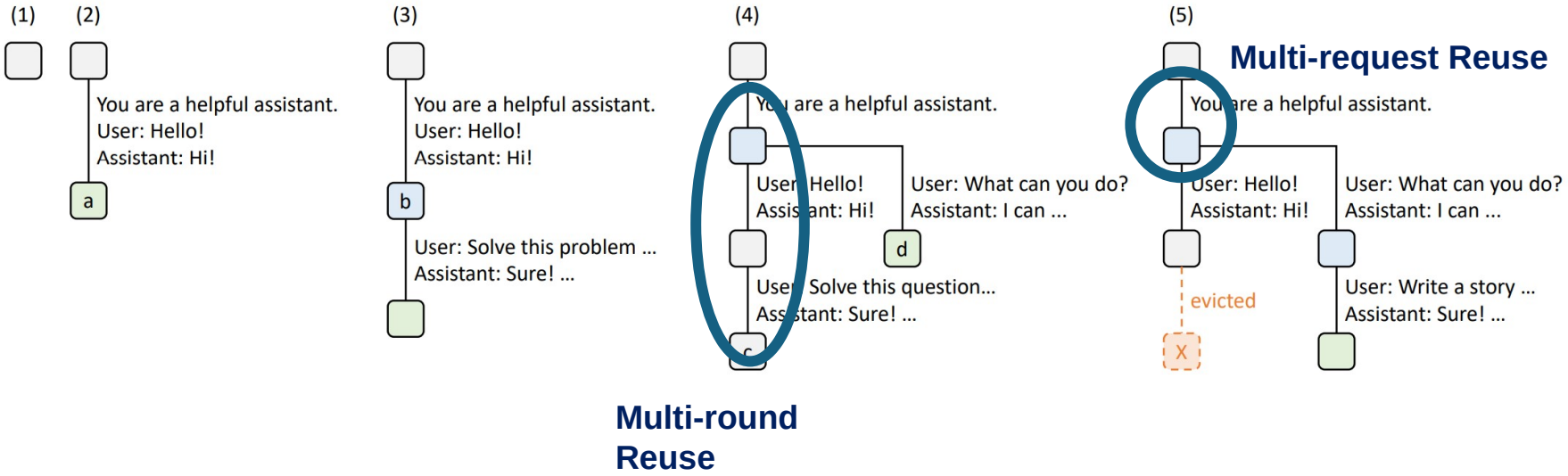
What if these two pages are the same?



Shared pages only need to prefill once and reduces both memory capacity and bandwidth in decoding

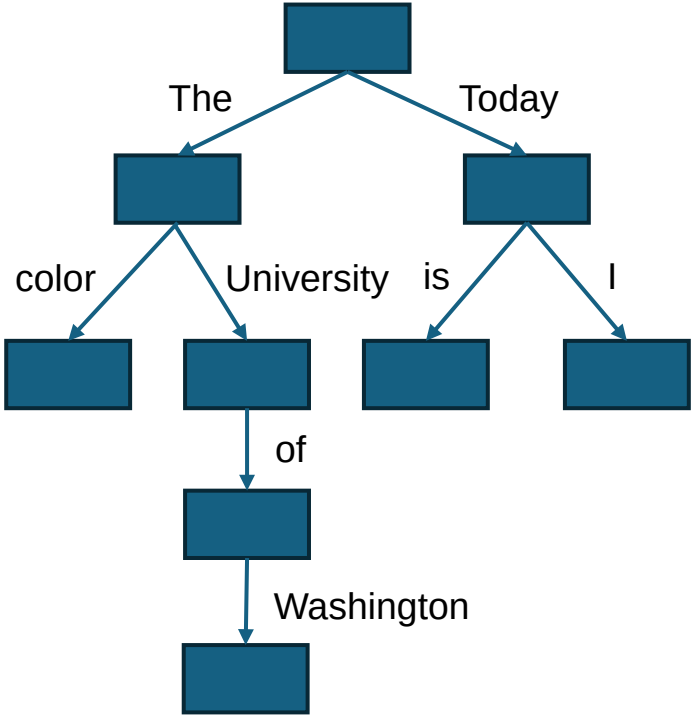
Prefix Tree

1. You are a helpful assistant. User:Hello, Assistant:Hi!
2. You are a helpful assistant. User:Hello, Assistant:Hi!, User: Solve this problem…
3. You are a helpful assistant. User:What can you do?



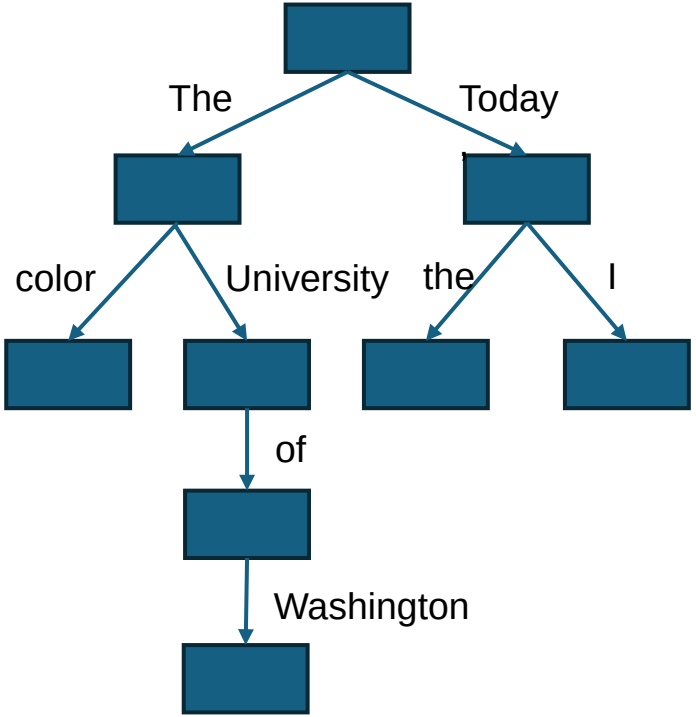
Zheng, Lianmin et al. “SGLang: Efficient Execution of Structured Language Model Programs.” *Neural Information Processing Systems* (2023).

Prefix Tree – Identify Prefix



The University of Washington is

Prefix Tree– Identify Prefix



Today, we will

Batched Decoding Exploiting a Common Prefix

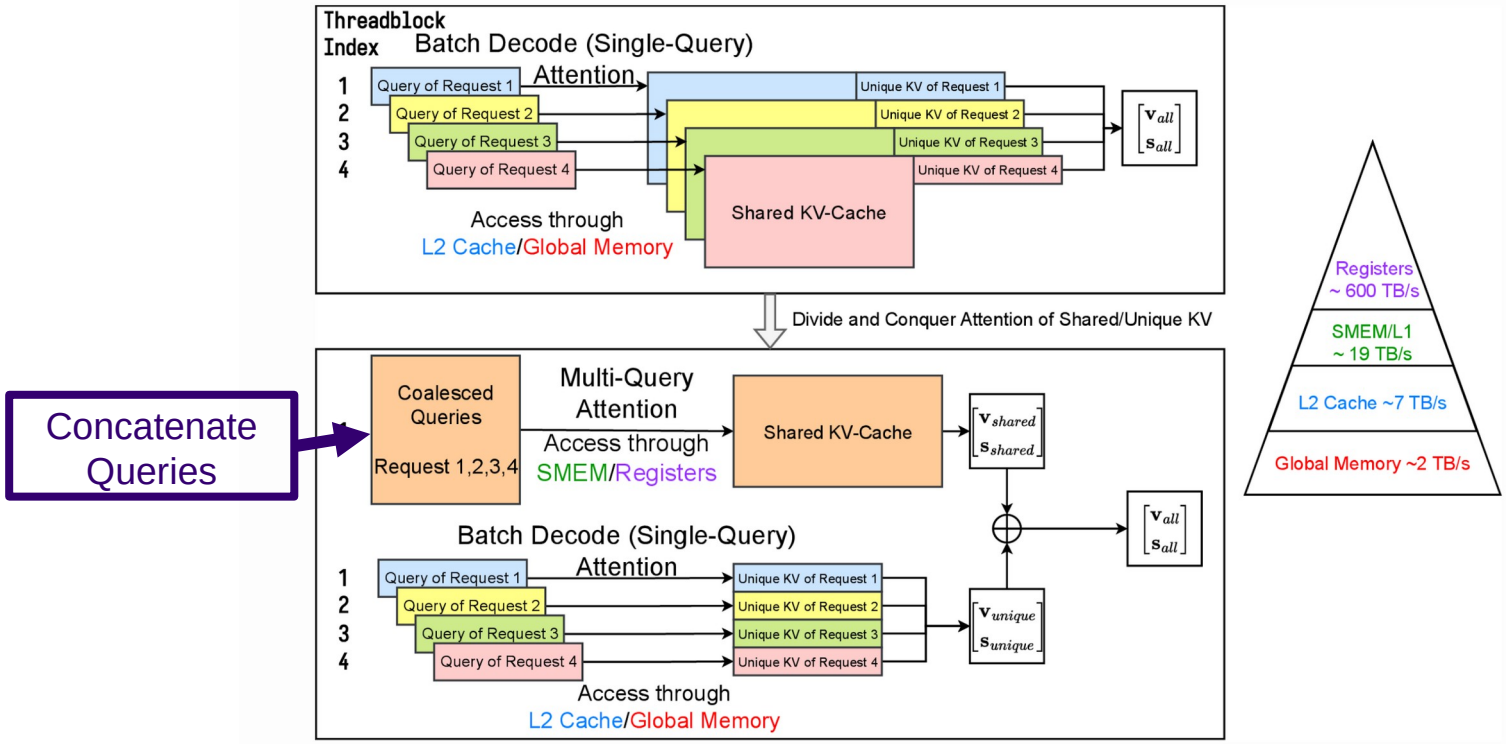


Figure 3. Workflow of Cascade Inference, throughput values adapted from blog: TPU vs GPU vs Cerebras vs Graphcore: A Fair Comparison between ML Hardware

https://flashinfer.ai/2024/02/02/cascade-inference.html

Performance gain of prefix sharing

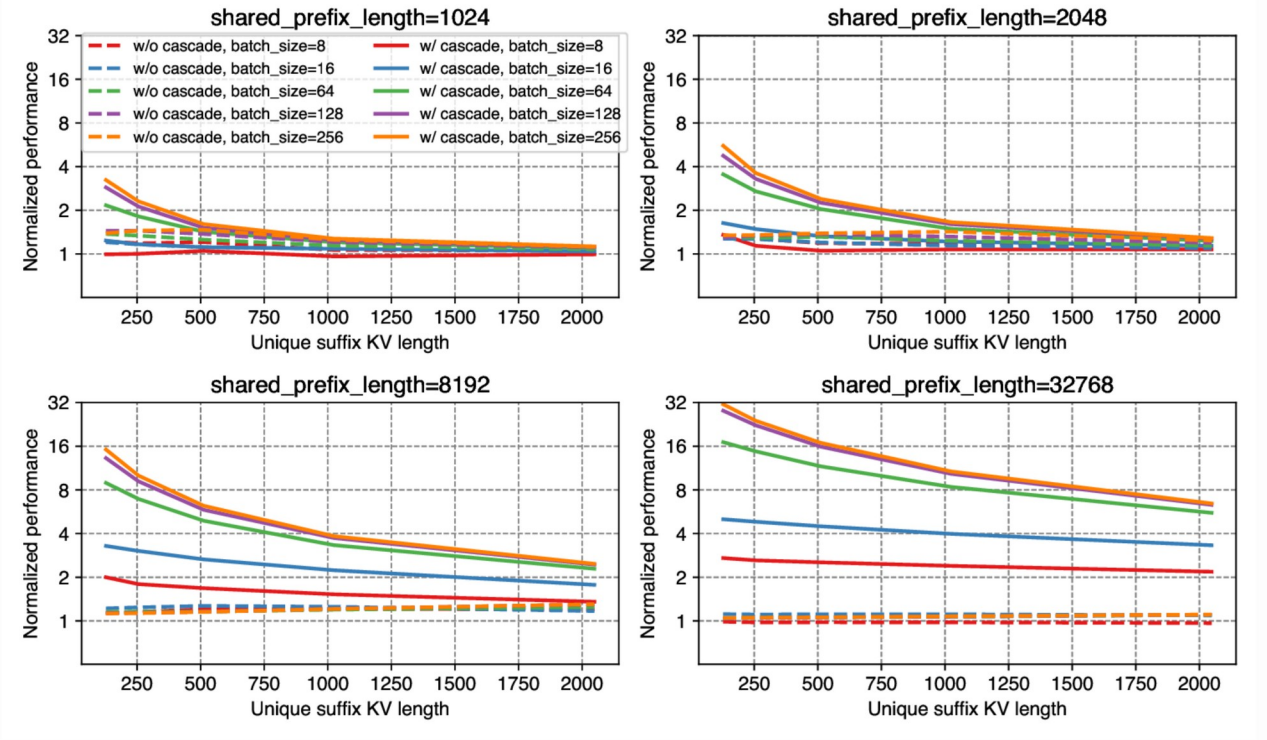


Figure 4. Speedup over vLLM PageAttention on H100 SXM 80GB

https://flashinfer.ai/2024/02/02/cascade-inference.html

Prefix Sharing Pros and Cons

The prefix with length p shared across n requests

- Reduced prefill computation (n*p -> p)
- Reduced KV cache size (n*p -> p)
- Reduced memory loading during decoding (n*p -> p)
- The matching can be performed asynchronously.
  - When a new request comes
- Need to store prefix chunks while some of them are not reused -> wasted GPU memory

Recomputing vs Load/Offload

- When the GPU memory is not enough to hold the Radix Tree:
  - Evict and recompute when needed
  - Send to CPU memory and load back when needed
- Recomputation (p, prefix cache length):
  - $T_{recompute} = \frac{2pP_{model}}{Compute}$
- Load:
  - $T_{load} = \frac{2 * Stype * \frac{D_{model}}{GQA} * L * p}{PCIE_Bandwidth}$

$$Throughput = \frac{Compute}{2P_{model}}$$

Recomputing vs Load/Offload

$$T_{recompute} = \frac{2pP_{model}}{Compute}$$

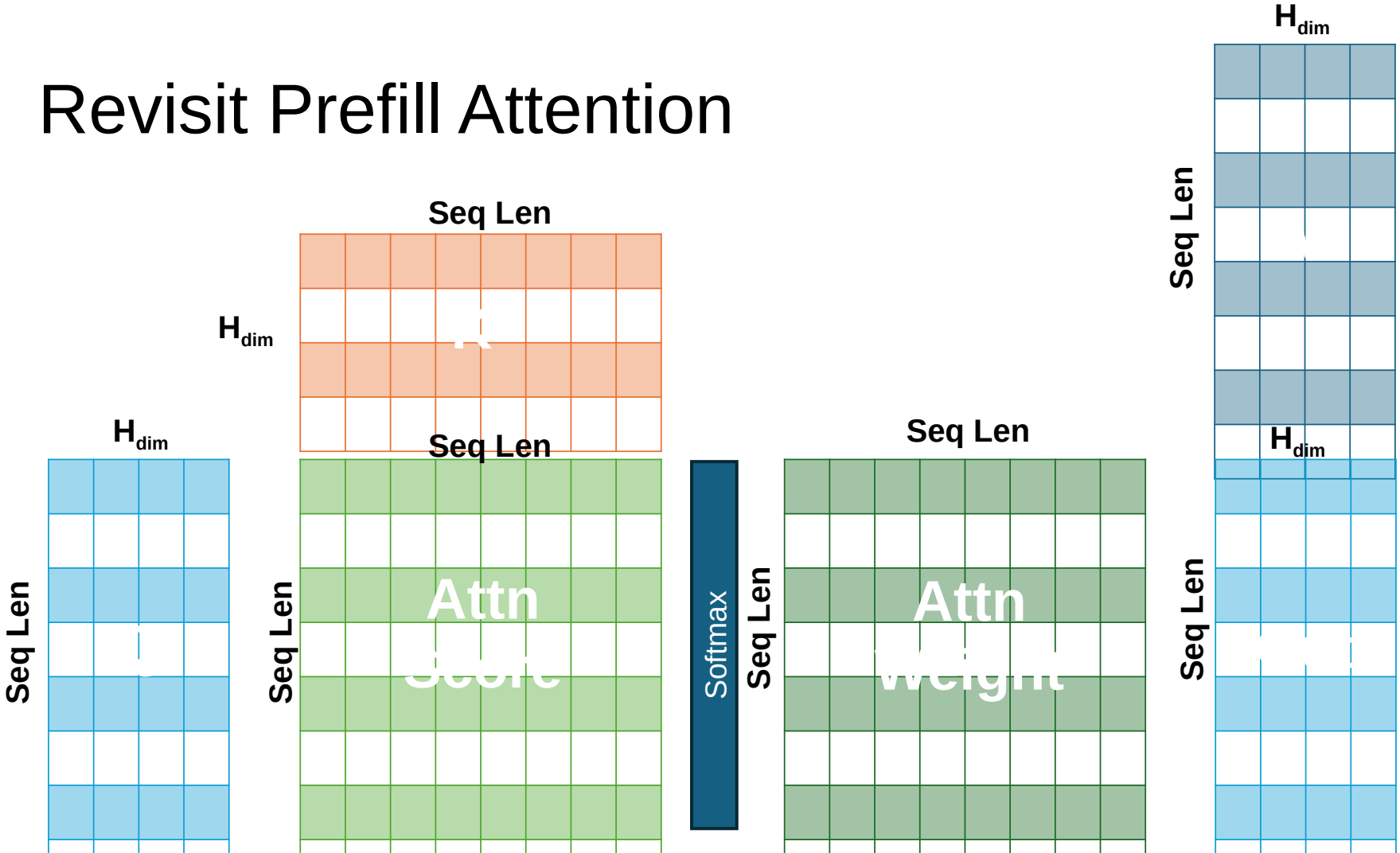
$$T_{load} = \frac{2*dtype*\frac{D_{model}}{GQA}*L*p}{PCIE_Bandwidth}$$

$$\frac{T_{recompute}}{T_{load}} = \frac{PCIE_{Bandwidth}\ P_{compute}}{dtype*\frac{D_{model}}{GQA}*L*Compute}$$

A100: 30G * 8 *10^9 / (2 * 1024 * 32 * 300T) = 12

Loading is much faster than recomputing

Revisit Prefill Attention



Softmax Numerical Stability

$$sm(x_i) = \frac{e^{x_i}}{\sum_{j=1}^{d} e^{x_j}}$$

```
x = np.array([10, 2, 10000, 4])
print(softmax(x))
```

Too large!

```
output: [0.0,  0.0, nan,  0.0]
```

$$sm(x_i) = \frac{e^{x_i - c}}{\sum_{j=1}^{d} e^{x_j - c}}$$

Usually c=max(xi)

```
x = np.array([10, 2, 10000, 4])
print(softmax(x))
```

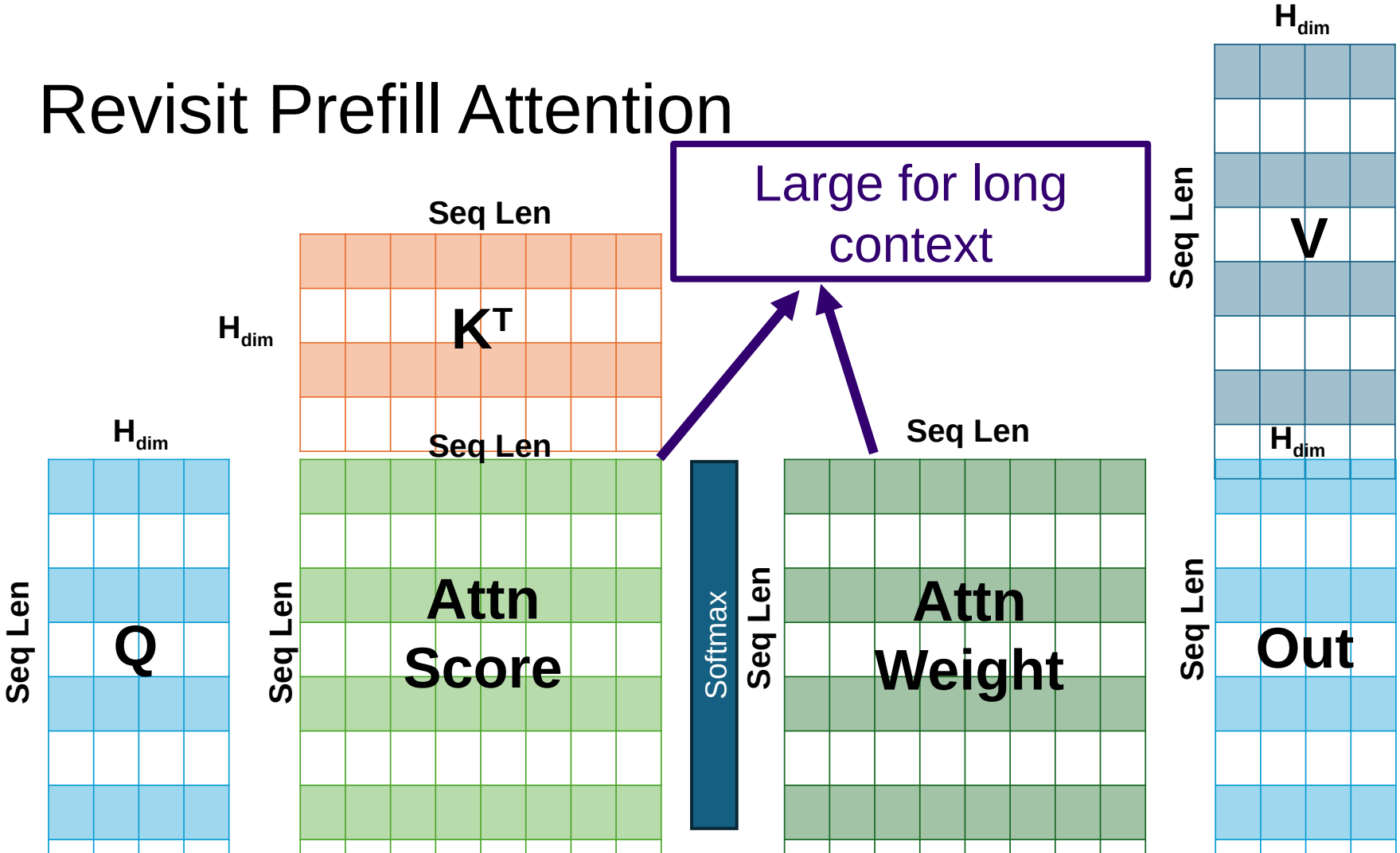
```
output: [0., 0., 1., 0.]
```

https://ogunlao.github.io/2020/04/26/you_dont_really_know_softmax.html

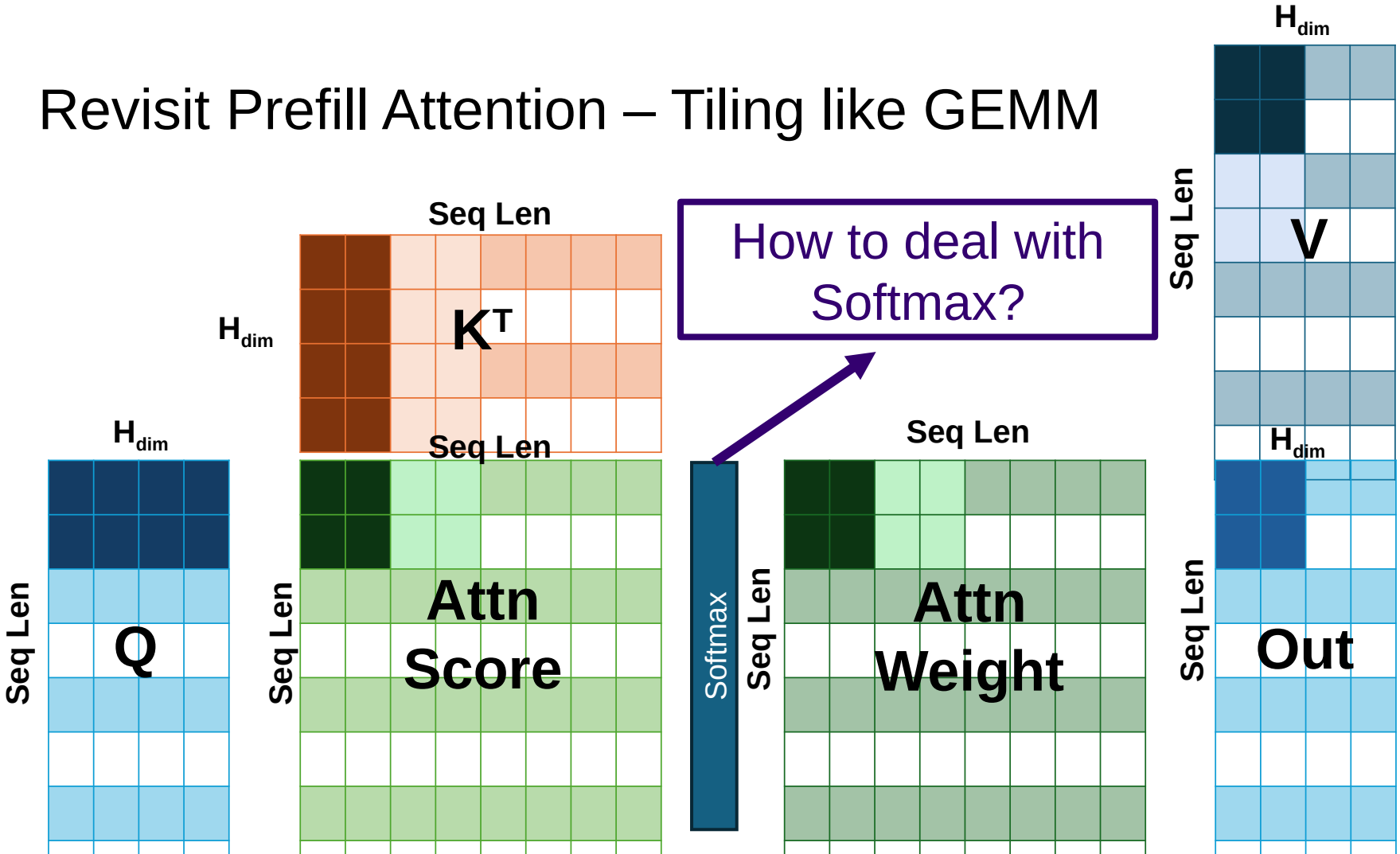
Softmax Numerical Stability

```
def softmax(x):
    max_x = np.max(x)
    exp_x = np.exp(x - max_x)
    sum_exp_x = np.sum(exp_x)
    sm_x = exp_x/sum_exp_x
    return sm_x
```

Revisit Prefill Attention



Revisit Prefill Attention – Tiling like GEMM



Original Attention Calculation:

1	2	1	-2
1	2	-1	0
1	2	1	1
1	2	1	0

$$sm(x_i) = \frac{e^{x_i - c}}{\sum_{j=1}^{d} e^{x_j - c}}$$

1	2
3	4
2	-2
-3	1

/ S1

2	2	2	2
2	-2	4	-2

4	8	2	-1
1	2	3	0

RowMax = 8

RowMax = 3

S1 = $e^{4-8} + e^{8-8} + e^{2-8} + e^{-1-8}$

e^{4-8}	e^{8-8}	e^{2-8}	e^{-1-8}
e^{1-3}	e^{2-3}	e^{3-3}	e^{0-3}

/ S1

/ S2

S2 = $e^{1-3} + e^{2-3} + e^{3-3} + e^{0-3}$

$1e^{4-8}+3e^{8-8}+2e^{2-8}-3e^{-1-8}$	$2e^{4-8}+4e^{8-8}-2e^{2-8}+e^{-1-8}$
$1e^{1-3}+3e^{2-3}+2e^{3-3}-3e^{0-3}$	$2e^{1-3}+4e^{2-3}-2e^{3-3}+e^{0-3}$

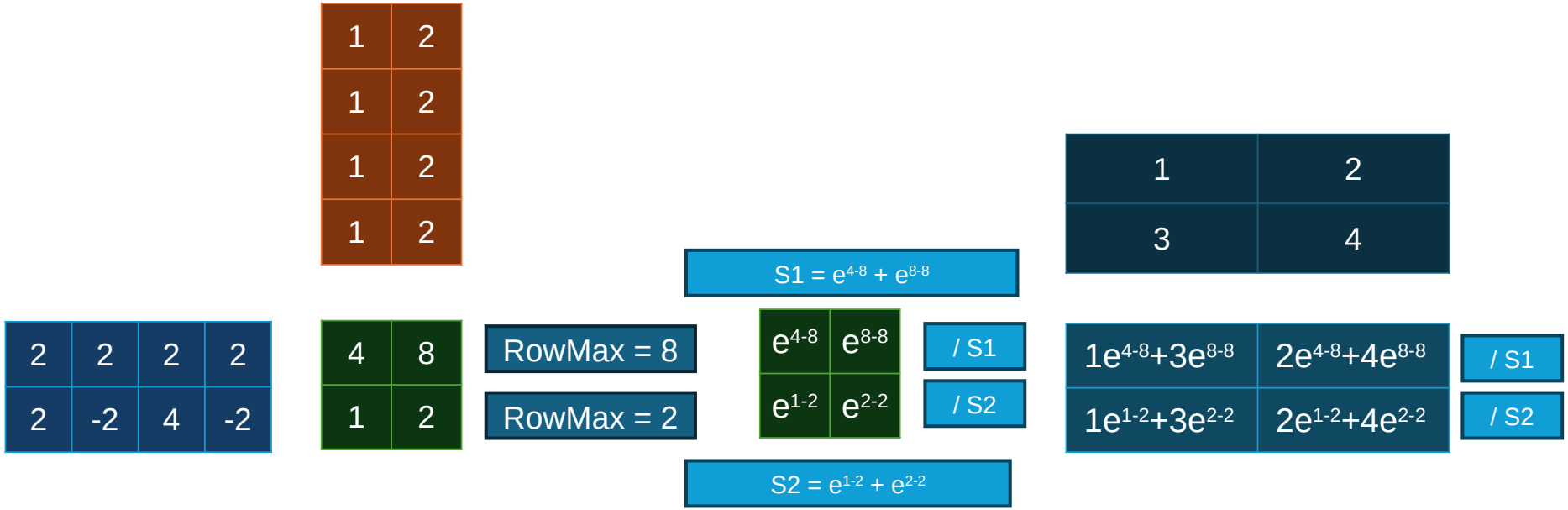
/ S2

$Q*K^T/sqrt(D)$

Stable Softmax

Softmax * V

FlashAttn – Calculating the first block separately



$Q*K^T/sqrt(D)$ | Stable Softmax | Softmax * V

Dao et al. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness

FlashAttn – Calculating the second block separately

1	-2
-1	0
1	1
1	0

2	2	2	2
2	-2	4	-2

2	-1
3	0

RowMax = 2

RowMax = 3

$S1 = e^{2-2} + e^{-1-2}$

e^{2-2}	e^{-1-2}	/ S1
e^{3-3}	e^{0-3}	/ S2

$S2 = e^{3-3} + e^{0-3}$

2	-2
-3	1

$2e^{2-2}-3e^{-1-2}$	$-2e^{2-2}+e^{-1-2}$	/ S1
$2e^{3-3}-3e^{0-3}$	$-2e^{3-3}+e^{0-3}$	/ S2

$Q*K^T/sqrt(D)$

Stable Softmax

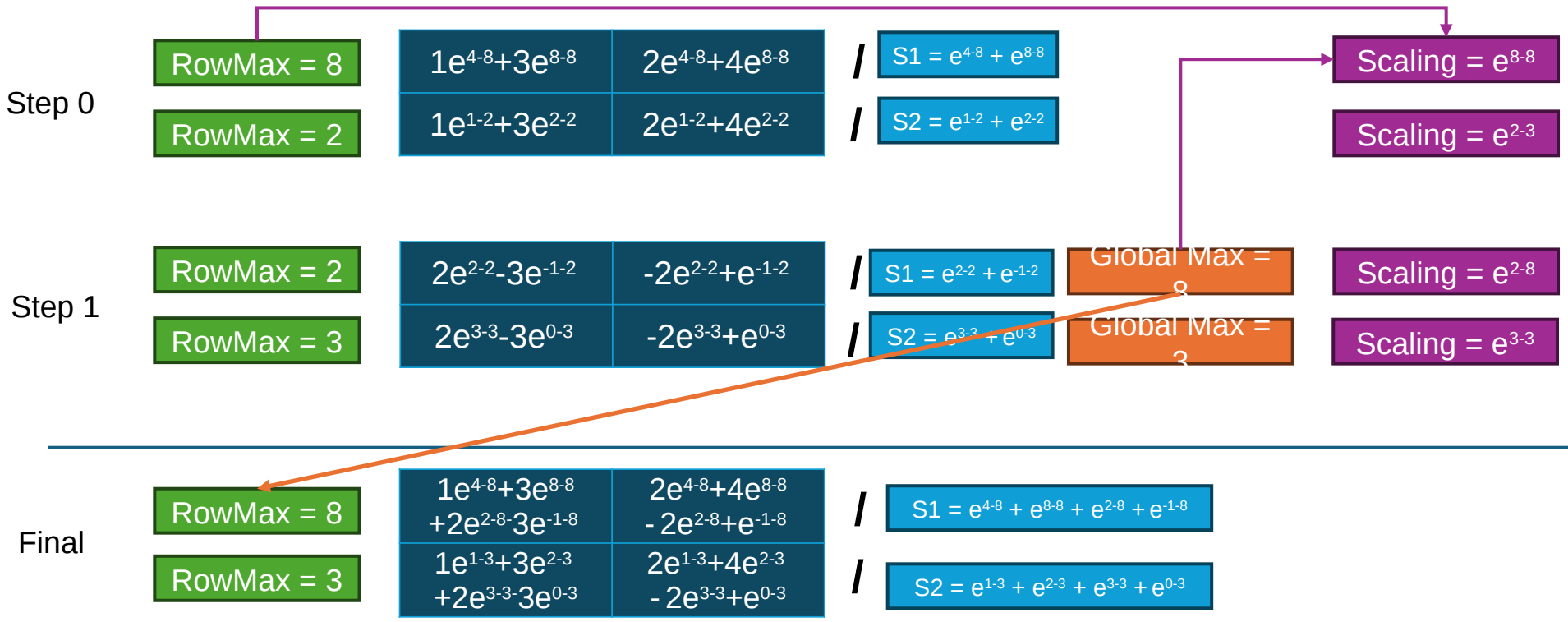
Softmax * V

FlashAttn – Combine two blocks

Step 0	RowMax = 8	$1e^{4-8}+3e^{8-8}$	$2e^{4-8}+4e^{8-8}$	/	$S1 = e^{4-8} + e^{8-8}$
	RowMax = 2	$1e^{1-2}+3e^{2-2}$	$2e^{1-2}+4e^{2-2}$	/	$S2 = e^{1-2} + e^{2-2}$
Step 1	RowMax = 2	$2e^{2-2}-3e^{-1-2}$	$-1e^{2-2}-e^{-1-2}$	/	$S1 = e^{2-2} + e^{-1-2}$
	RowMax = 3	$2e^{3-3}-3e^{0-3}$	$-2e^{3-3}+e^{0-3}$	/	$S2 = e^{3-3} + e^{0-3}$
Final	RowMax = 8	$1e^{4-8}+3e^{8-8}$ $+2e^{2-8}-3e^{-1-8}$	$2e^{4-8}+4e^{8-8}$ $-1e^{2-8}-e^{-1-8}$	/	$S1 = e^{4-8} + e^{8-8} + e^{2-8} + e^{-1-8}$
	RowMax = 3	$1e^{1-3}+3e^{2-3}$ $+2e^{3-3}-3e^{0-3}$	$2e^{1-3}+4e^{2-3}$ $-2e^{3-3}+e^{0-3}$	/	$S2 = e^{1-3} + e^{2-3} + e^{3-3} + e^{0-3}$

Similar but have wrong scaling

FlashAttn – Rescaling numerator and denominator



FlashAttn – Rescaling numerator

Step 0

Scaling = e^{8-8}	$1e^{4-8}+3e^{8-8}$	$2e^{4-8}+4e^{8-8}$
Scaling = e^{2-3}	$1e^{1-2}+3e^{2-2}$	$2e^{1-2}+4e^{2-2}$

$e^{8-8}(1e^{4-8}+3e^{8-8})$	$e^{8-8}(2e^{4-8}+4e^{8-8})$
$e^{2-3}(1e^{1-2}+3e^{2-2})$	$e^{2-3}(2e^{1-2}+4e^{2-2})$

Step 1

Scaling = e^{2-8}	$2e^{2-2}-3e^{-1-2}$	$-2e^{2-2}+e^{-1-2}$
Scaling = e^{3-3}	$2e^{3-3}-3e^{0-3}$	$-2e^{3-3}+e^{0-3}$

$e^{2-8}(2e^{2-2}-3e^{-1-2})$	$e^{2-8}(-2e^{2-2}+e^{-1-2})$
$e^{3-3}(2e^{3-3}-3e^{0-3})$	$e^{3-3}(-2e^{3-3}+e^{0-3})$

Final

$1e^{4-8}+3e^{8-8}$ $+2e^{2-8}-3e^{-1-8}$	$2e^{4-8}+4e^{8-8}$ $-2e^{2-8}+e^{-1-8}$
$1e^{1-3}+3e^{2-3}$ $+2e^{3-3}-3e^{0-3}$	$2e^{1-3}+4e^{2-3}$ $-2e^{3-3}+e^{0-3}$

/ $S1 = e^{4-8} + e^{8-8} + e^{2-8} + e^{-1-8}$

/ $S2 = e^{1-3} + e^{2-3} + e^{3-3} + e^{0-3}$

FlashAttn – Rescaling the Numerator

Step 0

$e^{8-8}(1e^{4-8}+3e^{8-8})$	$e^{8-8}(2e^{4-8}+4e^{8-8})$
$e^{2-3}(1e^{1-2}+3e^{2-2})$	$e^{2-3}(2e^{1-2}+4e^{2-2})$

$1e^{4-8}+3e^{8-8}$	$2e^{4-8}+4e^{8-8}$
$1e^{1-3}+3e^{2-3}$	$2e^{1-3}+4e^{2-3}$

+

Step 1

$e^{2-8}(2e^{2-2}-3e^{-1-2})$	$e^{2-8}(-2e^{2-2}+e^{-1-2})$
$e^{3-3}(2e^{3-3}-3e^{0-3})$	$e^{3-3}(-2e^{3-3}+e^{0-3})$

$2e^{2-8}-3e^{-1-8}$	$-2e^{2-8}+e^{-1-8}$
$2e^{3-3}-3e^{0-3}$	$-2e^{3-3}+e^{0-3}$

Final

$1e^{4-8}+3e^{8-8}$ $+2e^{2-8}-3e^{-1-8}$	$2e^{4-8}+4e^{8-8}$ $-2e^{2-8}+e^{-1-8}$
$1e^{1-3}+3e^{2-3}$ $+2e^{3-3}-3e^{0-3}$	$2e^{1-3}+4e^{2-3}$ $-2e^{3-3}+e^{0-3}$

FlashAttn – Rescaling the Denominator

Step 0

$\text{Scaling} = e^{8-8}$

$\text{Scaling} = e^{2-3}$

$S1 = e^{4-8} + e^{8-8}$

$S2 = e^{1-2} + e^{2-2}$

$S1 = e^{8-8}\,(e^{4-8} + e^{8-8})$

$S2 = e^{2-3}\,(e^{1-2} + e^{2-2})$

$S1 = e^{4-8} + e^{8-8}$

$S2 = e^{1-3} + e^{2-3}$

+

Step 1

$\text{Scaling} = e^{2-8}$

$\text{Scaling} = e^{3-3}$

$S1 = e^{2-2} + e^{-1-2}$

$S2 = e^{3-3} + e^{0-3}$

$S1 = e^{2-8}\,(e^{2-2} + e^{-1-2})$

$S2 = e^{3-3}\,(e^{3-3} + e^{0-3})$

$S1 = e^{2-8} + e^{-1-8}$

$S2 = e^{3-3} + e^{0-3}$

Final

$S1 = e^{4-8} + e^{8-8} + e^{2-8} + e^{-1-8}$

$S2 = e^{1-3} + e^{2-3} + e^{3-3} + e^{0-3}$

FlashAttn – Combine two blocks

Step 0	RowMax = 8	$1e^{4-8}+3e^{8-8}$	$2e^{4-8}+4e^{8-8}$	$S1 = e^{4-8} + e^{8-8}$
	RowMax = 2	$1e^{1-3}+3e^{2-3}$	$2e^{1-3}+4e^{2-3}$	$S2 = e^{1-3} + e^{2-3}$
Step 1	RowMax = 2	$2e^{2-8}-3e^{-1-8}$	$-2e^{2-8}+e^{-1-8}$	$S1 = e^{2-8} + e^{-1-8}$
	RowMax = 3	$2e^{3-3}-3e^{0-3}$	$-2e^{3-3}+e^{0-3}$	$S2 = e^{3-3} + e^{0-3}$

Final	RowMax = 8	$1e^{4-8}+3e^{8-8}$ $+2e^{2-8}-3e^{-1-8}$	$2e^{4-8}+4e^{8-8}$ $-2e^{2-8}+e^{-1-8}$	/	$S1 = e^{4-8} + e^{8-8} + e^{2-8} + e^{-1-8}$
	RowMax = 3	$1e^{1-3}+3e^{2-3}$ $+2e^{3-3}-3e^{0-3}$	$2e^{1-3}+4e^{2-3}$ $-2e^{3-3}+e^{0-3}$	/	$S2 = e^{1-3} + e^{2-3} + e^{3-3} + e^{0-3}$

FlashAttn

2	2	2	2
2	-2	4	-2

1	2
1	2
1	2
1	2

4	8
1	2

RowMax = 8

RowMax = 2

Global Max = 8

Global Max = 2

Keep

e^{4-8}	e^{8-8}
e^{1-2}	e^{2-2}

$S1 = e^{4-8} + e^{8-8}$

$S2 = e^{1-2} + e^{2-2}$

Keep

1	2
3	4

$1e^{4-8}+3e^{8-8}$	$2e^{4-8}+4e^{8-8}$
$1e^{1-2}+3e^{2-2}$	$2e^{1-2}+4e^{2-2}$

Keep

FlashAttn

$1e^{4-8}+3e^{8-8}$	$2e^{4-8}+4e^{8-8}$
$1e^{1-2}+3e^{2-2}$	$2e^{1-2}+4e^{2-2}$

$1e^{4-8}+3e^{8-8}$	$2e^{4-8}+4e^{8-8}$
$1e^{1-3}+3e^{2-3}$	$2e^{1-3}+4e^{2-3}$

1	-2
-1	0
1	1
1	0

Global Max = 8

Global Max = 2

2	-2
-3	1

2	2	2	2
2	-2	4	-2

2	-1
3	0

RowMax = 2

RowMax = 3

Global Max = 8

Global Max = 3

Keep

e^{2-8}	e^{-1-8}
e^{3-3}	e^{0-3}

$2e^{2-8}-3e^{-1-8}$	$-2e^{2-8}+e^{-1-8}$
$2e^{3-3}-3e^{0-3}$	$-2e^{3-3}+e^{0-3}$

$S1 = e^{4-8} + e^{8-8}$

$S2 = e^{1-2} + e^{2-2}$

$S1 = e^{2-8} + e^{-1-8}$

$S2 = e^{3-3} + e^{0-3}$

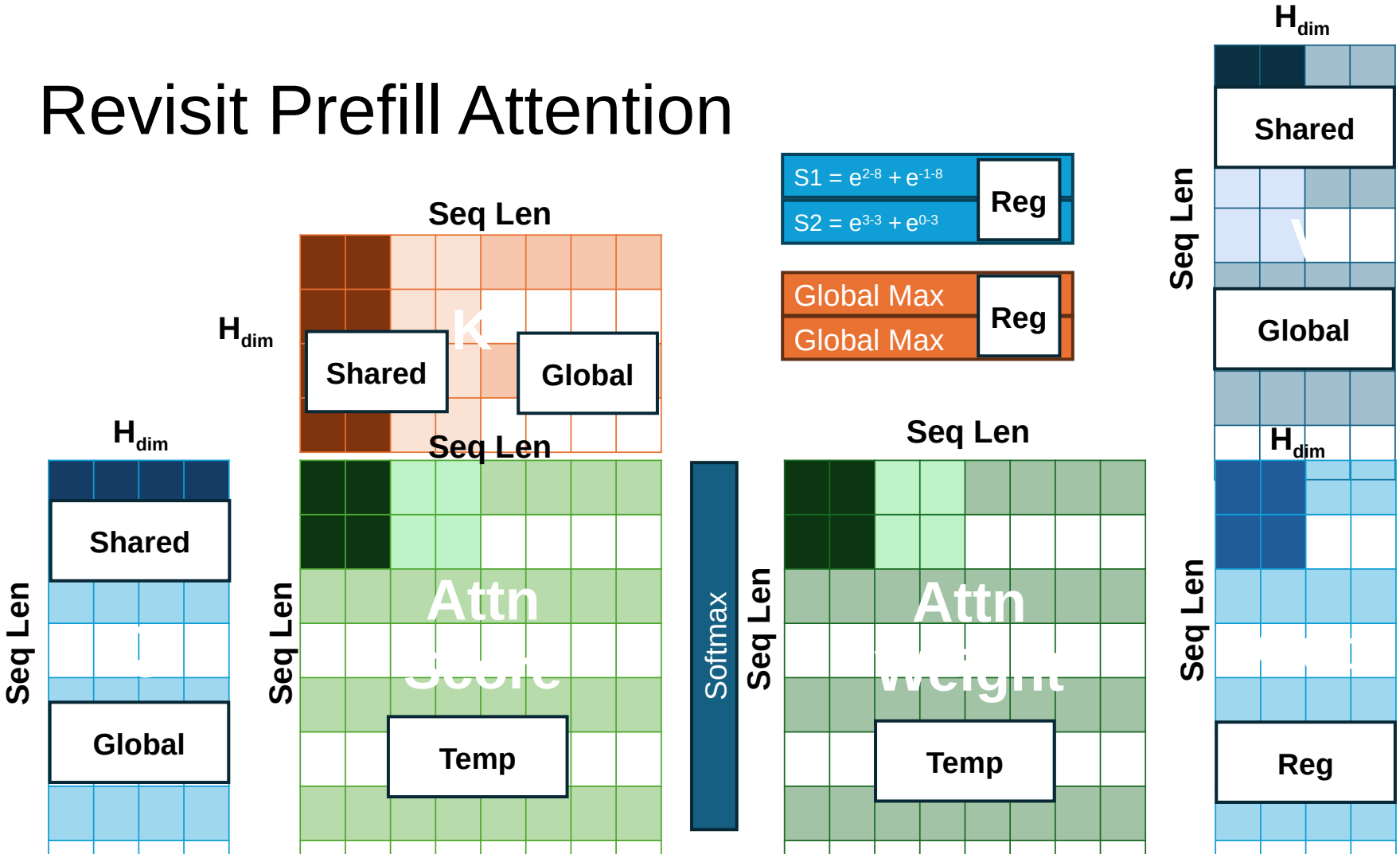
$S1 = e^{4-8} + e^{8-8} + e^{2-8} + e^{-1-8}$

$S2 = e^{1-3} + e^{2-3} + e^{3-3} + e^{0-3}$

Keep

$1e^{4-8}+3e^{8-8}$ $+2e^{2-8}-3e^{-1-8}$	$2e^{4-8}+4e^{8-8}$ $-2e^{2-8}+e^{-1-8}$
$1e^{1-3}+3e^{2-3}$ $+2e^{3-3}-3e^{0-3}$	$2e^{1-3}+4e^{2-3}$ $-2e^{3-3}+e^{0-3}$

Revisit Prefill Attention



Causal Mask

2	2	2	2
2	-2	4	-2

1	2
1	2
1	2
1	2

4	-Inf
1	2

RowMax = 4

RowMax = 2

Global Max = 4

Global Max = 2

Keep

e^{4-4}	0
e^{1-2}	e^{2-2}

$S1 = e^{4-4}$

$S2 = e^{1-2} + e^{2-2}$

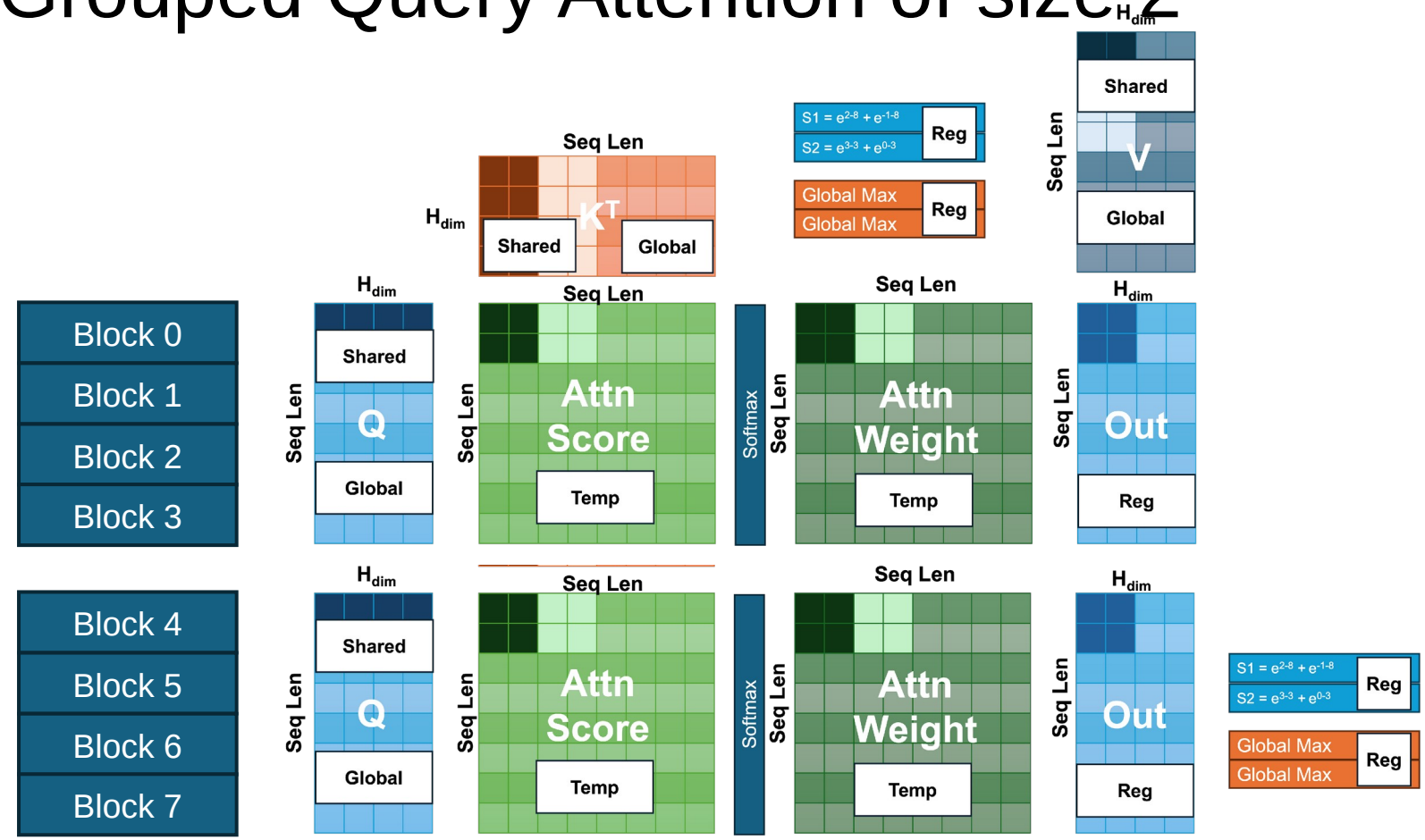
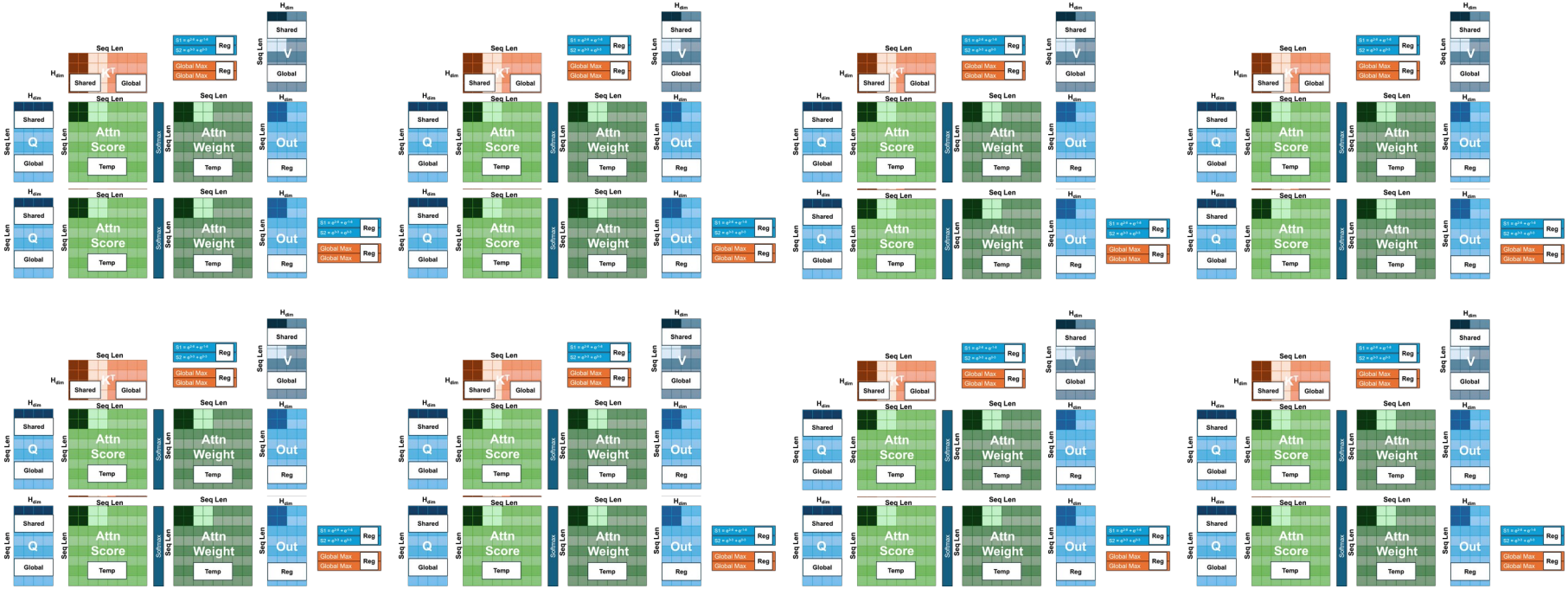
Keep

1	2
3	4

$1e^{4-4}$	$2e^{4-4}$
$1e^{1-2}+3e^{2-2}$	$2e^{1-2}+4e^{2-2}$

Keep

Grouped Query Attention of size 2



Expanding to all KV heads